



ANTHROPIC

AI 最佳实践 统一指南



底层原则



通用提示技术



让模型“思考”



工具使用与 Agent



上下文工程



安全与护栏



测试与评估



编程特殊实践



常见陷阱与反模式



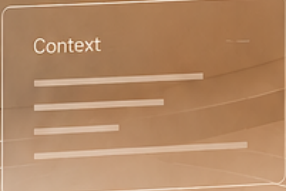
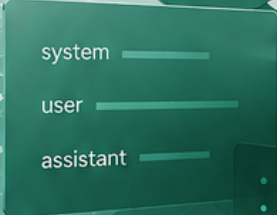
分歧点



工业级提示模板



递进能力



——基于 OpenAI 与 Anthropic 官方文档的整合

本文档整合并系统化两家公司关于"如何用好 AI"的全部核心官方文章。结构按**底层原则**而非按公司组织，目的是找出两家共通的本质、不同处的取舍，以及在不同场景下能直接照用的具体做法。

0. 资料来源

OpenAI

- 《A Practical Guide to Building Agents》（OpenAI 官方业务白皮书 PDF）
- 《Prompt engineering》（OpenAI API 文档）
- 《GPT-5 prompting guide》（OpenAI Cookbook）
- 《GPT-5.1 prompting guide》（OpenAI Cookbook）
- 《Prompt guidance》（GPT-5.4 / 5.5 提示指南）
- 《Reasoning best practices》（OpenAI API 文档）
- 《Prompt engineering best practices for ChatGPT》（OpenAI Help Center）
- OpenAI Academy 提示工程资源

Anthropic

- 《Prompting best practices》（Claude API 文档，活动版核心参考）
- 《Prompt engineering overview》
- 《Building effective agents》（Anthropic 工程博客）
- 《Building agents with the Claude Agent SDK》
- 《Claude Code best practices》
- 《Writing effective tools for agents》
- 《Effective harnesses for long-running agents》
- 《Equipping agents for the real world with Agent Skills》
- 《Define success criteria and build evaluations》（Strengthen guardrails 系列）

1. 底层原则：为什么提示要这样写

两家公司在所有文档里反复强调的，是三条无法绕开的事实。

1.1 模型不会读心。 OpenAI 的"清晰指令"和 Anthropic 的"清晰直接"说的是同一件事：模型不知道你脑子里的隐含假设。Anthropic 给出了一个判断标准——把你的提示拿给一个对任务背景没有了解的同事看，如果他会困惑，模型也会。

1.2 把模型当作"聪明但缺乏组织背景的新同事"。 这是两家不约而同使用的类比。新同事再聪明，也不知道你的命名规范、业务流程、客户偏好。所以**信息密度比口令更重要**：与其反复强调"重要！"，不如把它需要知道的事实和约束直接写出来。

1.3 提示工程是迭代的，不是一次性的。 三个先决条件必须先于"调提示"完成：

1. 清晰的成功标准（什么算成功？）
2. 经验性的测试方法（怎么测？）
3. 一个初稿提示（先有再优化）

没有评估，所有"调优"都是凭感觉。Anthropic 把这点放在所有 prompt engineering 文档的开头，OpenAI 把它作为六大策略

2. 通用提示技术（与具体模型无关）

这一章是无论用 GPT 还是 Claude、无论是 API 还是聊天界面都适用的核心技术。

2.1 清晰直接的指令

说"做什么", 而非"不要做什么"。Anthropic 明确这一点; OpenAI 在格式控制中给出相同建议。

- 较弱: 不要用 markdown 回答。
- 较好: 用流畅的散文段落回答。

指定输出的具体属性: 长度、格式、结构、风格、语气、目标受众。OpenAI 给的反例非常直观——"写一篇关于市场营销的文章"远不如"写一篇 500 字、面向中小企业主、聚焦内容营销的文章"。

顺序步骤用编号或符号列。 当顺序或完整性重要时, 把指令拆成可清点的步骤。

2.2 提供上下文与动机

不要只给指令, 给"为什么"。Anthropic 的原文: 模型聪明到可以从解释中泛化。

- 较弱: 永远先回答用户问题再问澄清问题。
- 较好: 先回答用户问题再问澄清问题。我们的用户在等待时容易流失, 先回答能减少跳出。

把任务发生的环境(用户是谁、要给谁看、什么用途)写进去, 几乎总能让输出更准。

2.3 使用示例 (Few-Shot / Multishot)

两家公司一致认为示例是改变输出最稳的杠杆。Anthropic 建议**3–5 个示例**, 并强调三点:

- **相关性:** 示例要贴近真实用例
- **多样性:** 覆盖边界情况, 且差异够大以免模型抓到不相关的共性
- **结构化:** 用 `<example>` 标签(多个用 `<examples>`) 和指令分开

OpenAI 在 Help Center 推荐**少样本胜过零样本**, 并展示了如何用 `<product_review>` 和 `<assistant_response>` 配对的方式把样例喂给模型。

何时不要用示例: 当使用推理模型 (OpenAI 的 o 系列) 时, 先试零样本, 往往就够了; 如果加示例, 要确保示例和指令高度一致, 否则会有反效果。

2.4 结构化标记: XML 标签 + Markdown

这是两家高度趋同的做法。

Anthropic 把 XML 标签作为头号推荐技术, 原因是 Claude 训练过程中大量使用 XML。常见模式:

```
<instructions>...</instructions>
<context>...</context>
<input>...</input>
<examples>
  <example>...</example>
</examples>
```

OpenAI 在 GPT-5 之后明确推荐组合用法: Markdown 标题划分大节、XML 标签界定可引用的内容片段。一份典型的开发者消息 (developer message) 按这个顺序组织:

- **Identity (身份):** 助手的角色、风格、目标

- **Instructions** (指令)：要做什么、不要做什么
- **Examples** (示例)：输入与期望输出
- **Context** (上下文)：私有数据、相关信息（通常放在最后）

为什么 XML 比纯文本好：当提示混合了指令、示例、用户输入、参考材料时，模型容易把它们混在一起处理。标签是给模型的“这是什么类型的内容”信号。

重要细节 (Anthropic)：标签命名要一致；嵌套有自然层级时使用嵌套（`<documents>` 里放 `<document index="1">`）。

2.5 角色设定

在系统/开发者消息中设置角色，即使只有一句话也有用。

```
You are a helpful coding assistant specializing in Python.
```

Anthropic 的额外提醒：**角色不要过度约束**——过于狭窄的角色定义会让模型在边缘情况下死板。

2.6 长上下文的特殊处理 (20k+ tokens)

当你要传入大量参考资料（合同、代码库、研究文档），位置和结构对效果影响巨大。

Anthropic 的三条具体做法：

1. 长文档放最前面，查询/指令/示例放后面。在测试中这一调整能把响应质量提升最多 30%。
2. 多文档用 XML 包裹：

```
<document index="1">
  <source>q3_earnings.pdf</source>
  <document_content>...</document_content>
</document>
```

3. 让模型先引用相关段落，再回答。这能帮模型从噪声里聚焦到真正相关的信息。

OpenAI 的对应做法：把会反复使用的内容放在提示的最前面、API 请求 JSON 体的最前面，以最大化 **prompt caching** 带来的成本和延迟节省。

2.7 控制输出格式

四种方式，从粗到细：

1. **明确告诉模型要什么**：“用流畅段落”而非“不要用列表”
2. **用 XML 格式指示符**：`<smoothly_flowng_prose_paragraphs>` 标签
3. **让提示风格匹配输出风格**：提示里少用 markdown，输出就少有 markdown
4. **详细规范**：长篇指令说明何时用什么格式（见 Anthropic 的 `<avoid_excessive_markdown_and_bullet_points>` 模板）

OpenAI 在 API 中的默认行为：GPT-5 系列在 API 中默认不输出 **markdown**（保留兼容性）。要让它输出 markdown，需在开发者消息首行加 `Formatting re-enabled` 或显式指示。

2.8 让指令优先级清晰

OpenAI 的指令链 (chain of command)：

- `system` / `developer` 消息：应用程序设定的规则与业务逻辑——像函数定义
- `user` 消息：终端用户的输入——像函数参数
- `assistant` 消息：模型生成的内容

更高优先级的指令凌驾更低优先级——这是为什么把核心规则放进 `developer/system` 消息能抵抗用户提示注入。

3. 让模型"思考"：推理控制

这是两家近几代模型的核心差异化，也是用户最容易用错的地方。

3.1 推理模型 vs 非推理（GPT）模型

OpenAI 给出了非常清晰的二分法：

维度	GPT 模型	推理模型（o 系列 / GPT-5 reasoning）
类比	资历浅的同事	资深同事
适合	明确定义、需要执行的任务	复杂、需判断、含歧义的问题
提示风格	精确、详细、规定步骤	高层目标，让它自己想办法
速度/成本	快、便宜	慢、贵
何时选	速度+成本最重要	准确性+可靠性最重要

最常见的反模式：用对推理模型的方法去提示 GPT 模型（太笼统），或反过来（手把手 step-by-step 反而干扰推理）。

3.2 推理深度参数

两家都有"推理力度"的旋钮：

- **OpenAI**: reasoning_effort — minimal / low / medium / high
- **Anthropic**: effort — low / medium / high / xhigh / max

两家共通建议：

- 执行型工作流（字段抽取、分类、轻量转换）→ 低力度起步
- 研究型工作流（长上下文综合、多文档审查、策略撰写）→ 中或高
- 编程与 Agent 任务 → 高或最高
- 当看到"过度思考"（简单任务也磨蹭）→ 调低
- 当看到"思考不足"（复杂问题答得太浅）→ 调高，**不要靠提示绕**

Anthropic 的明确警告：Claude Opus 4.7 在低力度下会严格按字面执行，不会"善解人意"地泛化指令。需要广泛适用时要明说范围（"对每一节都应用此格式，不只是第一节"）。

3.3 思考触发的可引导性

如果模型思考过多（大型系统提示常导致此问题），可加入引导：

Thinking adds latency and should only be used when it will meaningfully improve answer quality – typically for problems that require multi-step reasoning. When in doubt, respond directly.

反向，若想强制思考：

This task involves multi-step reasoning. Think carefully through the problem before responding.

3.4 推理模型的关键提示原则（OpenAI）

- **不要写 CoT 提示：**"让我们一步步思考"对推理模型无用甚至有害——它内部已经在思考
- **保持简短直接：**高层目标 > 详细步骤
- **先零样本，需要时再加少样本**

- **明确约束**："预算控制在 \$500 以下"
- **明确成功标准**：让它知道什么算完成

4. 工具使用与 Agent 构建

这一章是 2024–2026 年这两年文档增长最多的部分，因为 Agent 才是真正发挥模型价值的场景。

4.1 何时该构建 Agent

OpenAI 在《A Practical Guide to Building Agents》中给出的三类高价值场景：

1. **复杂决策**：需要细微判断、例外处理、上下文敏感决定（如客服中的退款审批）
2. **难以维护的规则**：规则集变得庞杂、更新成本高（如供应商安全审查）
3. **大量非结构化数据**：自然语言理解、文档信息抽取、对话式交互（如保险理赔处理）

反过来：如果一个确定性方案就够用，就别上 Agent。Agent 用延迟和成本换取处理模糊性的能力。

4.2 Anthropic 的本质区分：工作流 vs Agent

- **工作流 (Workflow)**：LLM 和工具通过预定义的代码路径编排
- **Agent**：LLM 动态决定自己的流程和工具使用

Anthropic 反复强调一点（这是他们和很多框架供应商的核心分歧）：**最成功的实现是简单可组合的模式，不是复杂框架**。如果一个增强型 LLM（带检索、工具、记忆）能搞定，就不要上 Agent。

4.3 五种核心模式 (Anthropic)

1. **Prompt Chaining (提示链)**：固定步骤的串联，每步输出是下一步输入。适合：翻译、生成-评审-改写、规范化流程。
2. **Routing (路由)**：对不同类型输入分发到不同的专门提示/模型。适合：客服分类、把简单查询走小模型、复杂查询走大模型。
3. **Parallelization (并行化)**：两个变体——
 - **分块 (Sectioning)**：把任务切成独立子任务并行跑
 - **投票 (Voting)**：同一任务跑多次、合并结果增加置信度
4. **Orchestrator-Workers (编排者-工人)**：一个 LLM 动态拆解任务、把子任务分配给"工人"LLM。适合复杂度无法预先确定的任务，如跨多文件的代码修改。
5. **Evaluator-Optimizer (评估者-优化者)**：一个 LLM 生成、另一个评估并提供反馈，循环改进。适合有明确评估标准、迭代改进有价值的任务（如翻译润色、文学搜索）。

4.4 单 Agent vs 多 Agent (OpenAI)

OpenAI 的强烈建议：先把单 Agent 做到极致，再考虑拆。多 Agent 直觉上更"模块化"，但带来协调和评估开销。

何时该拆：

- **复杂逻辑**：当提示里有大量 if-then-else 分支，模板难以为继时
- **工具过载**：当工具数量超过 10-15 个、或工具之间相似/重叠让模型选错时

两种多 Agent 模式：

- **Manager 模式**：一个中央 Manager Agent 通过工具调用协调多个专门 Agent。适合需要一个 Agent 统筹用户体验的场景。
- **Decentralized 模式**：Agent 之间互相 handoff（一次性转交执行权）。适合分诊类、专门 Agent 完全接管的场景。

4.5 工具设计的最佳实践 (Anthropic 工程博客)

工具的质量决定 Agent 的天花板。

给工具描述的原则：把工具描述当成"给新员工的入职说明"。把你隐性带入的语境（特殊查询格式、术语定义、资源之间的关系）显式写出来。

命名要无歧义：用 `user_id` 而非 `user`；用 `customer_email` 而非 `email`。

输入输出要严格类型化：用严格的数据模型而非自由文本。

限制工具响应大小：Claude Code 默认把工具响应限制在 25,000 tokens。实现 pagination、范围选择、过滤、截断。

错误响应也要做提示工程：错误消息要明确说出"应该改什么、怎么改"，而非堆栈跟踪。

Anthropic 的元方法：用 Claude 本身来优化工具。建一个评估集，让 Claude 改写工具描述，比对结果——他们的实践显示这种循环能带来戏剧性提升。

4.6 并行工具调用

最新一代模型擅长并行工具调用。要刺激这一行为：

```
If you intend to call multiple tools and there are no dependencies between the tool calls, make all of the independent tool calls in parallel. Prioritize calling tools simultaneously whenever the actions can be done in parallel rather than sequentially. However, if some tool calls depend on previous calls to inform dependent values like the parameters, do NOT call these tools in parallel and instead call them sequentially.
```

4.7 控制 Agent 的"积极性"（GPT-5 / Claude 4.6+）

新一代模型的一个反复出现问题是**过度积极**——会探索过多、生成不必要的脚手架、做过多额外工作。

降低积极性（OpenAI 的 `<context_gathering>` 模板）：

```
Goal: Get enough context fast. Parallelize discovery and stop as soon as you can act.
```

Method:

- Start broad, then fan out to focused subqueries.
- Avoid over searching for context.

Early stop criteria:

- You can name exact content to change.
- Top hits converge (~70%) on one area/path.

Loop:

- Batch search → minimal plan → complete task.
- Search again only if validation fails.

提升积极性（OpenAI 的 `<persistence>` 模板）：

```
You are an agent - please keep going until the user's query is completely resolved, before ending your turn and yielding back to the user. Only terminate your turn when you are sure that the problem is solved. Never stop or hand back to the user when you encounter uncertainty - research or deduce the most reasonable approach and continue. Do not ask the human to confirm or clarify assumptions.
```

Anthropic 对应："Overeagerness" 和 "Avoid focusing on passing tests" 段落。具体做法：明确指出"不要无要求地添加额外功能、不要重构未被要求的代码、不要为不会发生的情况加防御代码"。

4.8 工具调用的"前奏"（Tool Preambles）

让 Agent 在调用工具前简要说明它要做什么、为什么——能极大改善长任务下的用户体验。

```
<tool_preambles>
```

- Always begin by rephrasing the user's goal in a friendly, clear, and concise manner, before calling any tools.
 - Then, immediately outline a structured plan detailing each logical step.
 - As you execute, narrate each step succinctly and sequentially.
 - Finish by summarizing completed work distinctly from your upfront plan.
- </tool_preambles>

5. 上下文工程（Context Engineering）

这是 Anthropic 在 2025 年下半年提出的核心概念：上下文不是越多越好；上下文的"信噪比"决定输出。

5.1 长上下文的腐败（Context Rot）

当上下文窗口被堆满，模型注意力会被稀释。两家做法分别是：

- **OpenAI**：compaction API、prompt caching、明确的 conversation state 管理
- **Anthropic**：上下文感知（模型自己知道还剩多少 token）、自适应思考、记忆工具

5.2 多上下文窗口工作流（Anthropic）

当任务跨越单个上下文窗口（如重构大型代码库），关键是让模型能从干净的窗口里恢复状态。

五个具体做法：

1. **第一个窗口用不同的提示**：用第一个上下文窗口做"初始化 Agent"，设置框架（写测试、创建启动脚本），后续窗口在此基础上迭代。
2. **让模型用结构化格式写测试**：让 Claude 在开始前创建测试并保存为 `tests.json`。明确告知"不能删除或修改测试，因为这可能导致功能缺失或 bug"。
3. **设置生活质量脚本**：让模型创建 `init.sh` 等启动脚本来优雅地启动服务、运行测试、运行 linter。
4. **优先开新窗口而非压缩**：Claude 4.5+ 极擅长从本地文件系统恢复状态。给它清晰的恢复指令：

```
- Call pwd; you can only read and write files in this directory.
- Review progress.txt, tests.json, and the git logs.
- Manually run through a fundamental integration test before
  implementing new features.
```

5. **提供验证工具**：随着自主任务时间增长，模型需要在没有人类反馈时验证自己（Playwright、computer use 等）。

5.3 状态管理的最佳实践

- **结构化数据用结构化格式**：测试结果、任务状态用 JSON
- **进度笔记用自由文本**：渐进式记录
- **用 git 跟踪状态**：git log 是天然的状态机
- **强调增量进度**：明确让模型每次做一小步、提交、然后继续

5.4 Skills：渐进式披露（Anthropic 已开源标准）

Skill 是组织好的文件夹（包含 SKILL.md 加可选的脚本、参考资料），Agent 可以动态发现和加载。设计核心是**渐进式披露**：

- 启动时只把每个 Skill 的名称和描述放入系统提示
- 模型决定何时需要时再加载完整内容
- 这意味着 Skill 内的内容可以无限大，因为不进基础上下文

何时建 Skill：

1. **通用能力增强**：模型开箱不擅长的事（生成 PDF、Excel、PowerPoint）
2. **组织/行业/个人工作流**：你的品牌指南、特定客户的合规要求

5.5 Agent Harness 的关键组件（Anthropic）

为长跑 Agent 构建好"环境"比调优提示更重要：

- 初始化 Agent：第一次运行时建立环境、写需求文档、设置测试
- 进度文件（如 `claude-progress.txt`）+ git 历史：让新会话快速理解任务状态
- 明确的开始/结束信号：模型知道一次会话该做什么、做完什么算完成

6. 安全与护栏（Guardrails）

6.1 多层防御（OpenAI）

OpenAI 强调"单一护栏不够"——构建多层组合：

护栏类型	作用
Relevance classifier	屏蔽离题输入（"帝国大厦多高"被电信客服 Agent 屏蔽）
Safety classifier	检测 jailbreak / 提示注入
PII filter	防止意外泄露个人信息
Moderation	仇恨言论、骚扰、暴力内容
Tool safeguards	给每个工具评估风险（读 vs 写、可逆 vs 不可逆、财务影响）
Rules-based protections	黑名单、长度限制、正则
Output validation	品牌一致性、内容合规

优化策略：

- 先关注数据隐私和内容安全
- 根据真实失败案例增加新护栏
- 在安全性和用户体验之间持续调优

6.2 平衡自主与安全（Anthropic）

Claude Opus 4.6 在无引导下可能采取难以撤销的动作。具体做法：

Consider the reversibility and potential impact of your actions. Encouraged to take local, reversible actions like editing files or running tests, but for actions that are hard to reverse, affect shared systems, or could be destructive, ask the user before proceeding.

Examples of actions that warrant confirmation:
- Destructive operations: deleting files, dropping tables, `rm -rf`
- Hard to reverse: `git push --force`, `git reset --hard`
- Operations visible to others: pushing code, commenting on PRs, sending messages

6.3 人工介入（Human in the Loop）

两类触发条件（OpenAI）：

- 超过失败阈值：连续多次未能理解用户意图 → 升级到人工
- 高风险动作：敏感、不可逆、高赌注（取消订单、大额退款、支付）→ 在 Agent 可靠性建立前强制人工

6.4 Prompt Injection 防御

- 把外部内容（文档、网页、用户上传文件）当作数据而非指令
- 用 XML 标签明确标记"这是参考材料"vs"这是指令"
- 系统级护栏检测明显的注入模式

7. 测试与评估

这是被两家公司放在最前面但被开发者最常跳过的部分。

7.1 先定义成功标准

在写提示之前，先回答：

- 这个任务的"对"是什么样子？
- 怎么测？（自动化指标 / LLM-as-judge / 人评）
- 失败的代价有多大？

Anthropic 在 prompt engineering overview 的第一节就要求三件事：清晰的成功标准、经验性的测试方法、初稿提示。

7.2 自评估让模型自我检查（两家共通）

让模型在给出最终答案前对照标准检查：

```
Before you finish, verify your answer against [test criteria].
```

OpenAI 给出的 "self-reflection" 模板（用于零到一应用生成）：

```
<self_reflection>
- First, think of a rubric until you are confident.
- Create a rubric with 5-7 categories. This is for your purposes only.
- Use the rubric to internally iterate. If your response is not hitting
  the top marks across all categories, you need to start again.
</self_reflection>
```

7.3 LLM as Judge

OpenAI 在 Reasoning best practices 里专门提到：用推理模型评估其他模型的输出是一个高价值用例。例如让 o1 评估 GPT-4o 的摘要质量，F1 分数从 0.12 提升到 0.74。

7.4 系统性改动而非"感觉"

修改提示后必须重测完整评估集，而非只看一两个例子。原因：

- 修复 case A 经常破坏 case B
- 个别"感觉变好"在统计意义上可能没显著差异
- 模型的非确定性让单次结果不可信

8. 编程场景的特殊实践

代码是当下最重要的 Agent 用例，两家都有大量针对性文档。

8.1 持久配置：CLAUDE.md / AGENTS.md

两家不约而同地推荐项目级长期记忆：

- Anthropic: `CLAUDE.md` (项目根目录及子目录可分层)
- OpenAI Codex: `AGENTS.md`

里面写什么：

- 项目约定（命名、目录结构、风格）
- 必跑的命令（启动、测试、linter）
- 关键的"陷阱"和反模式
- 团队共识和决策记录

8.2 编程任务的核心提示模板

Anthropic 推荐对编程 Agent 设置：

```
<investigate_before_answering>
Never speculate about code you have not opened. If the user references
a specific file, you MUST read the file before answering. Make sure to
investigate and read relevant files BEFORE answering questions.
</investigate_before_answering>
```

OpenAI (Terminal-Bench prompt 中提炼)：

```
<exploration>
Before coding, always:
- Decompose the request into explicit requirements, unclear areas,
  and hidden assumptions.
- Map the scope: identify codebase regions, files, functions, libraries.
- Check dependencies: frameworks, APIs, config files, data formats.
- Resolve ambiguity proactively.
- Define the output contract: exact deliverables.
- Formulate an execution plan in your own words.
</exploration>

<verification>
Routinely verify your code works as you progress, especially deliverables.
Don't hand back to the user until you are sure the problem is solved.
</verification>
```

8.3 避免"为了通过测试而 hack"

Anthropic 明确提示：

```
Please write a high-quality, general-purpose solution using the standard
tools available. Do not create helper scripts or workarounds. Implement
a solution that works correctly for all valid inputs, not just the test
cases. Do not hard-code values or create solutions that only work for
specific test inputs.
```

```
If the task is unreasonable or if any tests are incorrect, please inform
me rather than working around them.
```

8.4 避免过度工程

最常见的反模式之一。Anthropic 的明确提示：

```
Avoid over-engineering. Only make changes that are directly requested
or clearly necessary.

- Scope: Don't add features, refactor, or make "improvements" beyond
  what was asked.
- Documentation: Don't add docstrings or comments to code you didn't
  change.
```

- Defensive coding: Don't add error handling for scenarios that can't happen. Trust internal code guarantees. Only validate at system boundaries.
- Abstractions: Don't create helpers for one-time operations. Don't design for hypothetical future requirements.

8.5 前端设计避免"AI Slop"美学

两家在 2025 下半年都注意到模型在前端设计时会收敛到"通用 AI 味道"——紫色渐变、Inter 字体、千篇一律的卡片布局。

应对：

- 描述具体的视觉方向（不只是"现代"或"简洁"），具体到字体、色板、边距、阴影
- 或让模型在动手前提出 4 种不同方向让用户选
- Anthropic 把这种引导专门做成了一个 frontend-design Skill

8.6 多窗口编程 workflow

参考第 5.2 节。核心是让一个会话能优雅地把工作交给下一个会话，而不是想"一锤子搞定"。

9. 常见陷阱与反模式

把两家文档里"不要这么做"的内容汇总。

9.1 矛盾或模糊的指令

新一代模型（GPT-5 / Claude 4.6+）会**花费推理 token 去试图调和矛盾**，而不是随机选一个。OpenAI 在 GPT-5 prompting guide 里展示了一个客服 prompt 中三处隐性矛盾被解决后性能大幅提升的例子。

实用检查：定期把核心提示库拿给 GPT-5 / Claude 自己审查矛盾点。

9.2 过度提示

为旧版模型写的"务必！"、"非常重要！！!"、"如有疑问，用 X 工具"——这些在新模型上常导致**过度触发**。Anthropic 明确说：把强语气改成正常语气。Cursor 团队在与 OpenAI 合作调优 GPT-5 时也发现，去掉 "BE THOROUGH"、"FULL picture" 这类强调语让模型更聪明地决定何时探索、何时直接回答。

9.3 把 GPT 提示风格用在推理模型上

详细的 step-by-step 指令对推理模型是负担。**反过来**，对 GPT 模型只给高层目标会让它表现拉胯。

9.4 让模型猜代码

特别是当用户提到具体文件名时，让模型去读它而非猜——否则会产生看似合理的幻觉。

9.5 创建过多临时文件

Claude 4.5+ 在编程时会创建临时脚本作为"草稿纸"。如不希望这样：

```
If you create any temporary new files, scripts, or helper files for iteration, clean up these files by removing them at the end of the task.
```

9.6 上下文耗尽时草草收尾

让模型知道有压缩/记忆机制，不要因 token 焦虑提前结束：

Your context window will be automatically compacted as it approaches its limit, allowing you to continue working indefinitely from where you left off. Therefore, do not stop tasks early due to token budget concerns. As you approach your token budget limit, save your current progress and state to memory before the context window refreshes.

9.7 把所有需求一次塞给模型

在多步任务中前置说清楚胜过"边走边补"。Claude 4.7 的文档明确指出：模糊或欠规范的需求逐步给出，会降低性能和 token 效率。

10. 两家的分歧点

虽然 90% 的最佳实践趋同，仍有几处值得注意的差异。

10.1 推理模型的提示风格

- **OpenAI**: 对 o 系列推理模型，明确说"不要用 CoT 提示"、"保持简短"
- **Anthropic**: 仍鼓励让模型 think、用 `<thinking>` 标签结构化推理；但同样指出"general instructions 比 prescriptive steps 好"

实质上方向一致——给目标，让模型自己想——但表述风格不同。

10.2 Verbosity 控制

- **OpenAI**: GPT-5 引入了独立的 `verbosity` API 参数（区别于 `reasoning_effort`）
- **Anthropic**: 通过提示指令控制；Claude Opus 4.7 默认会根据任务复杂度自动校准长度

10.3 Prefilled Responses（预填回复）

- **OpenAI**: 未直接强调
- **Anthropic**: Claude 4.6 起取消支持对最后一条助手消息的预填——这个曾经常用的技术现已废弃，迁移到普通的指令式提示

10.4 Markdown 默认行为

- **OpenAI**: GPT-5 API 中默认不输出 `markdown`（兼容性），需要时显式开启
- **Anthropic**: Claude 默认会用 `markdown`，但有非常具体的指引（流畅散文 vs. 列表的选择）

10.5 框架取向

- **OpenAI**: 提供 Agents SDK、ChatKit、Agent Builder 等较完整的产品化栈
- **Anthropic**: 更倾向于"简单可组合的模式"，对复杂框架持保留态度（"成功实现都没用复杂框架"）

11. 核心提示模板速查

以下是可以直接套用的工业级提示片段（来自两家官方文档）。

11.1 通用清晰指令模板

```
# Identity
You are [角色], specialized in [领域]. Your goal is to [目标].

# Instructions
```

- [具体行为 1, 肯定语句而非否定]
- [具体行为 2]
- [边界条件和异常处理]

Examples

```
<example>
  <input>...</input>
  <output>...</output>
</example>
```

Context

[私有数据或参考材料]

11.2 长文档 RAG 模板

```
<documents>
  <document index="1">
    <source>filename.pdf</source>
    <document_content>...</document_content>
  </document>
  <document index="2">
    <source>filename2.pdf</source>
    <document_content>...</document_content>
  </document>
</documents>
```

[你的问题放在最后]

Before answering, quote the most relevant passages from each document inside <quotes> tags. Then answer based on those quotes.

11.3 Agent Persistence 模板

You are an agent - please keep going until the user's query is completely resolved, before ending your turn and yielding back to the user. Decompose the user's query into all required sub-requests, and confirm that each is completed. Do not stop after completing only part of the request. Only terminate your turn when you are sure the problem is solved.

You must plan extensively before each function call, and reflect on the outcomes of previous calls. Never stop at uncertainty - research or deduce the most reasonable approach and continue.

11.4 工具使用前奏模板

```
<tool_preambles>
- Always begin by rephrasing the user's goal in a clear, concise manner before calling any tools.
- Outline a structured plan detailing each logical step.
- As you execute, narrate each step succinctly.
- Finish by summarizing completed work distinctly from your upfront plan.
</tool_preambles>
```

11.5 并行工具调用模板

```
<use_parallel_tool_calls>
If you intend to call multiple tools and there are no dependencies between them, make all independent calls in parallel. Prioritize simultaneous calls whenever actions can be done in parallel rather than sequentially. However, if some calls depend on previous calls' results to inform parameters, do NOT call those in parallel. Never use placeholders or guess missing parameters.
</use_parallel_tool_calls>
```

11.6 上下文采集预算模板

```
<context_gathering>
Goal: Get enough context fast. Parallelize discovery and stop as soon
as you can act.

Method:
- Start broad, then fan out to focused subqueries.
- Launch varied queries in parallel; deduplicate paths.
- Avoid over searching.

Early stop:
- You can name exact content to change.
- Top hits converge (~70%) on one area.

Loop: Batch search → minimal plan → complete task.
Search again only if validation fails.
</context_gathering>
```

11.7 反过度工程模板

Avoid over-engineering. Only make changes directly requested or clearly necessary.

- Scope: No features, refactoring, or improvements beyond the request.
- Documentation: No docstrings or comments on code you didn't change.
- Defensive coding: No error handling for scenarios that can't happen.
- Abstractions: No helpers for one-time operations. No designing for hypothetical future requirements.

The right amount of complexity is the minimum needed for the current task.

11.8 自反思元提示（让模型优化自己的提示）

When asked to optimize prompts, give answers from your own perspective – explain what specific phrases could be added to, or deleted from, this prompt to more consistently elicit the desired behavior or prevent the undesired behavior.

Prompt: [PROMPT]

The desired behavior is for the agent to [DESIRED]; instead it [ACTUAL]. While keeping as much of the existing prompt intact as possible, what minimal edits would you make to more consistently address these shortcomings?

11.9 安全确认模板

Consider the reversibility and potential impact of your actions. Take local, reversible actions (editing files, running tests) freely. For actions that are hard to reverse, affect shared systems, or could be destructive, ask the user before proceeding.

Examples warranting confirmation:

- Destructive: deleting files, dropping tables, `rm -rf`
- Hard to reverse: `git push --force`, `git reset --hard`
- Visible to others: pushing code, commenting on PRs, sending messages, modifying shared infrastructure

Do not use destructive actions as shortcuts (e.g., `--no-verify`) or discard unfamiliar files that may be in-progress work.

12. 总结：从这两家文档里提炼的本质

把上百页材料浓缩成一句话：用 AI 用得好的关键不是"会用聪明的咒语"，而是"把你的真实意图和约束完整、清晰、结构化地传达给一个能理解你但不读心的合作者"。

更具体一点，是六条递进的能力：

1. **能定义成功**：你说不清"什么算好"，没人能帮你做出"好"——包括 AI。
2. **能写清指令**：把你脑子里的隐性背景显式化。模型是新同事，不是预言家。
3. **能选对模型与参数**：简单执行用快模型，复杂判断用推理模型；难度和 effort 匹配。
4. **能用工具与上下文**：模型本身是引擎，工具和上下文是燃料和路线图。
5. **能在 Agent 化时保持简单**：从最简单的增强型 LLM 起步，只在真正需要时增加复杂度。
6. **能持续测和迭代**：评估系统是 AI 工程化的真正壁垒，不是提示本身。

这两年的所有"prompt engineering 技巧"——XML 标签、少样本、CoT、思维树、ReAct、自反思——都是上述六条原则在不同细节场景下的展开。理解原则比记住具体咒语更重要。模型一直在变，原则不变。

本文档基于 2024–2026 年间 OpenAI 和 Anthropic 在其官方网站、文档站、博客和 Cookbook 上发布的关于 AI 使用最佳实践的核心文章整合而成。